

DATA ENGINEERING AND VISUALIZATION

ETLs, EDAs, Data Curation, and Visual Storytelling

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I



Outline

- 1 ETL and Exploratory Data Analysis
- 2 Data Curation
- 3 Data Visualization and Visual Storytelling



Outline

- 1 ETL and Exploratory Data Analysis
- 2 Data Curation
- 3 Data Visualization and Visual Storytelling

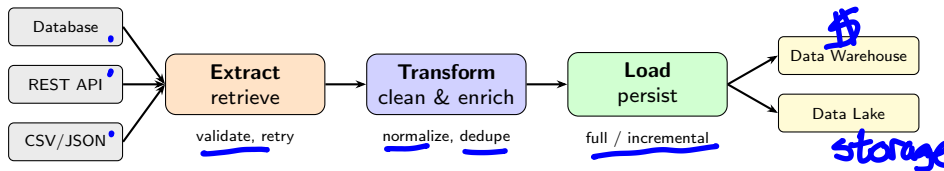


ETL: Extract, Transform, Load

ETL Pipeline

sequence of tasks

A data integration pattern that extracts raw data from heterogeneous sources, transforms it to meet quality and schema requirements, and loads it into a destination system ready for analysis.



- **Extract:** handle partial failures, pagination, and auth tokens *password*
- **Transform:** apply business rules, reshape schemas, quality checks
- **Load:** support upserts, a, and rollback on failure



- Controlled schema at load time
- Required with limited destination capacity
- Tools: Informatica, Talend

Load raw data **first**; transform inside the warehouse.

- Leverages cloud-scale compute
- Preserves raw data for replay
- Tools: dbt, Spark, BigQuery

- **Batch:** Apache Airflow (DAG), Luigi, Prefect
- **Stream:** Apache Kafka, Apache Flink, Spark Streaming
- **Cloud-native:** AWS Glue, Azure Data Factory, Google Dataflow

ETL vs. ELT: Modern Paradigm Shift

ETL (Classic)

Transform **before** loading.
Transforms run on a **dedicated server**.

- **Controlled** schema at load time
- Required with **limited** destination capacity
- Tools: Informatica, Talend

ELT (Modern)

Load raw data **first**; transform inside the warehouse.

- Leverages **cloud-scale** compute
- **Preserves raw** data for replay
- Tools: dbt, Spark, BigQuery

ETL Orchestration Tools

- **Batch:** Apache Airflow (DAG), Luigi, Prefect
- **Stream:** Apache Kafka, Apache Flink, Spark Streaming
- **Cloud-native:** AWS Glue, Azure Data Factory, Google Dataflow

ETL: Current Challenges

Core Engineering Challenges

Building reliable, scalable, and observable pipelines at production scale is significantly harder than it appears in tutorials.

- Schema drift: schemas change columns without notice $\Rightarrow$ pipelines silently break
- Data volume: terabyte-scale ingestion requires partitioning, parallelism, and incremental patterns
- Late-arriving data: events arrive out of order — watermarks and event-time windows required
- Idempotency: re-running a failed job must not produce duplicate records
- Observability: detect failures, measure performance, track state across pipeline stages
- Audit compliance: regulatory requirements require data retention, deletion, access auditing



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials

- **Schema drift**: source systems change columns **without notice** ⇒ pipelines silently break

- **Data volume**: terabyte-scale ingestion requires partitioning, parallelism, and incremental patterns

- **Late-arriving data**: events arrive out of order – watermarks and event time windows required

- **Idempotency**: re-running a failed job must not produce duplicate records

- **Observability**: detect failures, measure data freshness, track lineage across pipeline stages

- **PII compliance**: GDPR / CCPA require data masking, deletion, and access auditing



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials.

- **Schema drift**: source systems change columns **without notice** ⇒ pipelines silently break
- **Data volume**: terabyte-scale ingestion requires partitioning, parallelism, and incremental patterns
- **Late-arriving data**: events arrive **out of order** — watermarks and *event-time* windows required
- **Idempotency**: re-running a failed job must **not** produce duplicate records
- **Observability**: detect failures, measure **data freshness**, track lineage across pipeline stages
- **PII compliance**: **GDPR / CCPA** require data **masking**, deletion, and **access auditing**



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials.

- **Schema drift**: source systems change columns **without notice** $\Rightarrow$ pipelines silently break
- **Data volume**: **terabyte-scale** ingestion requires *partitioning*, *parallelism*, and **incremental** patterns
- **Late-arriving data**: events arrive **out of order** — watermarks and event-time windows required

- **Idempotency**: a failed job must not produce duplicate records
- **Observability**: detect failures, measure data freshness, track lineage across pipeline stages

PII compliance: GDPR / CCPA require data masking, deletion, and access auditing



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials.

- **Schema drift**: source systems change columns **without notice** $\Rightarrow$ pipelines silently break
- **Data volume**: **terabyte-scale** ingestion requires *partitioning*, *parallelism*, and **incremental** patterns
- **Late-arriving data**: events arrive **out of order** — watermarks and *event-time* windows required
- **Idempotency**: re-running a failed job must **not** produce **duplicate** records
- **Observability**: detect failures, measure **data freshness**, track **lineage** across pipeline stages
- **PII compliance**: **GDPR / CCPA** require data **masking**, **deletion**, and **access auditing**



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials.

- **Schema drift**: source systems change columns **without notice** $\Rightarrow$ pipelines silently break
- **Data volume**: **terabyte-scale** ingestion requires *partitioning*, *parallelism*, and **incremental** patterns
- **Late-arriving data**: events arrive **out of order** — watermarks and *event-time* windows required
- **Idempotency**: re-running a failed job must **not** produce duplicate records
- **Observability**: detect failures, measure **data freshness**, track **lineage** across pipeline stages

LAG

- **PII compliance**: **GDPR / CCPA** require data **masking**, **deletion**, and **access auditing**



ETL: Current Challenges

Core Engineering Challenges

Building **reliable**, **scalable**, and **observable** pipelines at production scale is significantly harder than it appears in tutorials.

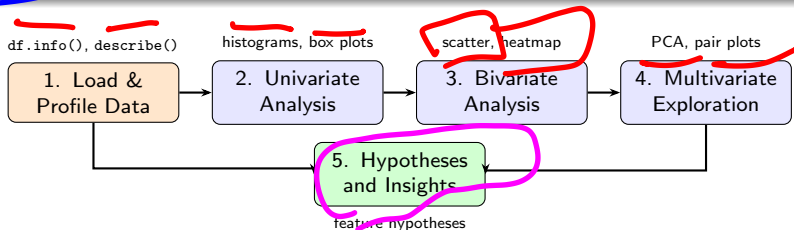
- **Schema drift**: source systems change columns **without notice** $\Rightarrow$ pipelines silently break
- **Data volume**: **terabyte-scale** ingestion requires *partitioning*, *parallelism*, and **incremental** patterns
- **Late-arriving data**: events arrive **out of order** — watermarks and *event-time* windows required
- **Idempotency**: re-running a failed job must **not** produce duplicate records
- **Observability**: detect failures, measure **data freshness**, track **lineage** across pipeline stages
- **PII compliance**: **GDPR** / **CCPA** require data **masking**, deletion, and **access auditing**



Exploratory Data Analysis (EDA)

Definition — Tukey (1977)

EDA is the practice of **examining datasets** to **summarize** their **main characteristics**, discover **patterns**, detect **anomalies**, and **generate hypotheses** before formal modelling.



• EDA is iterative: each analysis generates new questions

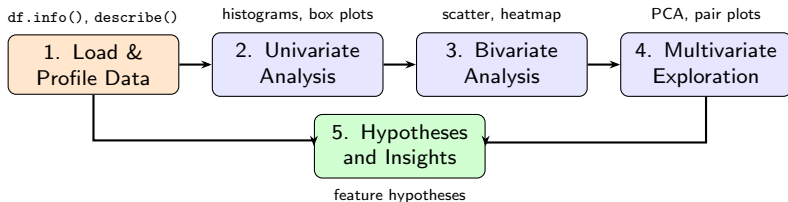
• Goal: understand distributions, relationships, and anomalies before any modelling



Exploratory Data Analysis (EDA)

Definition — Tukey (1977)

EDA is the practice of **examining datasets** to summarize their **main characteristics**, discover **patterns**, detect **anomalies**, and **generate hypotheses** before formal modelling.



- EDA is **iterative**. each analysis generates new *questions*
- Goal: understand **distributions**, *relationships*, and **anomalies** before any modelling



Descriptive Analytics: Central Tendency and Dispersion

Central Tendency

- **Mean:** $\bar{x} = \frac{1}{n} \sum x_i$ — **sensitive** to outliers
- **Median:** middle value; **robust** to outliers
- **Mode:** most frequent; useful for **categorical** data

Dispersion

- **Variance:**
 $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$
- **Std Dev:** $s = \sqrt{s^2}$; same units as data
- **IQR:** $Q_3 - Q_1$; **robust** spread measure
- **Range:** $x_{\max} - x_{\min}$; sensitive to extremes

Normal distribution — [≈68]-[≈95]-[≈99.7] rule



Descriptive Analytics: Central Tendency and Dispersion

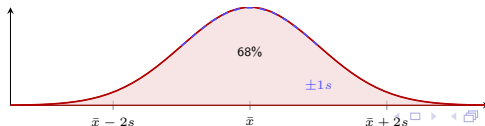
Central Tendency

- **Mean:** $\bar{x} = \frac{1}{n} \sum x_i$ — **sensitive** to outliers
- **Median:** middle value; **robust** to outliers
- **Mode:** most frequent; useful for **categorical** data

Dispersion

- **Variance:**
 $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$
- **Std Dev:** $s = \sqrt{s^2}$; same units as data
- **IQR:** $Q_3 - Q_1$; **robust** spread measure
- **Range:** $x_{\max} - x_{\min}$; sensitive to extremes

Normal distribution — [≈ 68]-[≈ 95]-[≈ 99.7] rule



Quantiles, Skewness, and Kurtosis

Quantiles

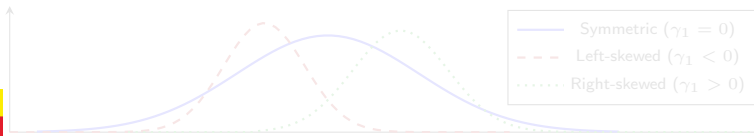
Values that **partition** a sorted dataset into equal groups.

- Q_1, Q_2, Q_3 : **quartiles**
- P_{95} : 95th **percentile** — SLA thresholds
- **IQR** = $Q_3 - Q_1$: robust **spread** measure

Skewness and Kurtosis

- $\gamma_1 > 0$: right-skewed (long right tail)
- $\gamma_1 < 0$: left-skewed (long left tail)
- $\gamma_2 > 0$: leptokurtic — heavy tails
- $\gamma_2 < 0$: platykurtic — light tails

Distribution shapes



Quantiles, Skewness, and Kurtosis

Quantiles

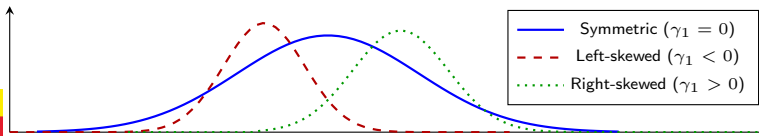
Values that **partition** a sorted dataset into equal groups.

- Q_1, Q_2, Q_3 : **quartiles**
- P_{95} : 95th **percentile** — SLA thresholds
- **IQR** = $Q_3 - Q_1$: robust **spread** measure

Skewness and Kurtosis

- $\gamma_1 > 0$: **right-skewed** (long right tail)
- $\gamma_1 < 0$: **left-skewed** (long left tail)
- $\gamma_2 > 0$: **leptokurtic** — heavy tails
- $\gamma_2 < 0$: **platykurtic** — light tails

Distribution shapes



Correlation and Association in EDA

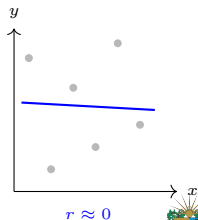
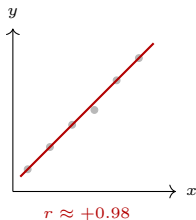
Pearson Correlation r

Measures **linear association** between two continuous variables, $r \in [-1, 1]$.

Correlation does not imply causation.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

- $|r| \geq 0.7$: **strong**
 $0.4 \leq |r| < 0.7$: **moderate**
- **Spearman ρ** : rank-based,
robust to outliers;
 monotonic relationship
- **Kendall τ** : concordant
 pairs; for **small samples** or
 many ties



Correlation and Association in EDA

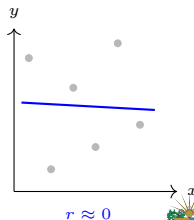
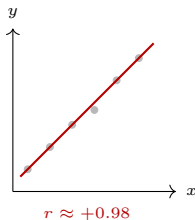
Pearson Correlation r

Measures **linear association** between two continuous variables, $r \in [-1, 1]$.

Correlation does not imply causation.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

- $|r| \geq 0.7$: **strong**
 $0.4 \leq |r| < 0.7$: **moderate**
- **Spearman ρ** : rank-based, **robust** to outliers; monotonic relationship
- **Kendall τ** : concordant pairs; for **small samples** or many ties



Correlation does not imply Causation

LA CORRELACIÓN NO IMPLICA CAUSALIDAD



EDA Applications: Connecting to the Pipeline

EDA as a Quality Gate

EDA is not just exploratory — it **validates pipeline outputs** at the **Silver layer** before Gold aggregations are computed.

What EDA reveals:

- **Distribution shifts** between ingestion runs
- Unexpected **null rates** in critical columns
- **Duplicate** records from concurrent DAG runs
- Temporal **gaps** — missing ingestion windows
- **Cardinality** anomalies in categorical columns

EDA tools (Python):

```
* pandas: .describe(),
      .value_counts(),
      .isnull().sum()
* ydata-profiling: auto full
      report
* seaborn: pairplot(),
      heatmap()
* plotly: interactive
      exploration
```



EDA Applications: Connecting to the Pipeline

EDA as a Quality Gate

EDA is not just exploratory — it **validates pipeline outputs** at the **Silver layer** before Gold aggregations are computed.

What EDA reveals:

- **Distribution shifts** between ingestion runs
- Unexpected **null rates** in critical columns
- **Duplicate** records from concurrent DAG runs
- Temporal **gaps** — missing ingestion windows
- **Cardinality** anomalies in categorical columns

EDA tools (Python):

- pandas: `.describe()`, `.value_counts()`, `.isnull().sum()`
- ydata-profiling: auto **full report**
- seaborn: `pairplot()`, `heatmap()`
- plotly: **interactive** exploration



Outline

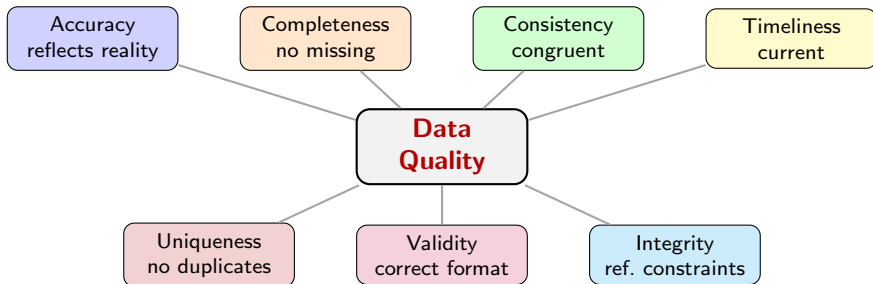
- 1 ETL and Exploratory Data Analysis
- 2 Data Curation
- 3 Data Visualization and Visual Storytelling



Data Quality Dimensions

“Garbage In, Garbage Out”

Poor data quality leads to **biased models**, *incorrect decisions*, and **wasted engineering resources**. Quality must be **measured**, not assumed.



Outlier Handling I: *Detection Methods*

Outlier

An observation that **deviates markedly** from other data points. May be a **measurement error**, a **rare genuine event**, or a **data quality issue** — context determines the treatment.

Statistical Methods:

- **Z-score:**
 $|z_i| > 3$; $z_i = (x_i - \mu)/\sigma$
 — assumes normality
- **IQR Rule:**
 $x < Q_1 - 1.5 \cdot \text{IQR}$ or
 $x > Q_3 + 1.5 \cdot \text{IQR}$
 — robust
- **Modified Z-score:** uses median instead of mean; better for skewed data

Algorithmic Methods:

- **Isolation Forest:** random splits; anomalies isolated in fewer partitions
- **DBSCAN:** density-based; points in low-density regions are outliers
- **Domain rules:** values outside business logic (e.g., age < 0, price < 0)



Outlier Handling I: *Detection Methods*

Outlier

An observation that **deviates markedly** from other data points. May be a **measurement error**, a **rare genuine event**, or a **data quality issue** — context determines the treatment.

Statistical Methods:

- **Z-score:**

$$|z_i| > 3; z_i = (x_i - \mu) / \sigma$$

— **assumes normality**

- **IQR Rule:**

$$x < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x > Q_3 + 1.5 \cdot \text{IQR}$$

— **robust**

- **Modified Z-score:** uses **median** instead of mean; better for skewed data

Algorithmic Methods:

- **Isolation Forest:** random splits; anomalies isolated in fewer partitions
- **DBSCAN:** density-based; points in low-density regions are outliers
- **Domain rules:** values outside business logic (e.g., age < 0, price < 0)



Outlier Handling I: *Detection Methods*

Outlier

An observation that **deviates markedly** from other data points. May be a **measurement error**, a **rare genuine event**, or a **data quality issue** — context determines the treatment.

Statistical Methods:

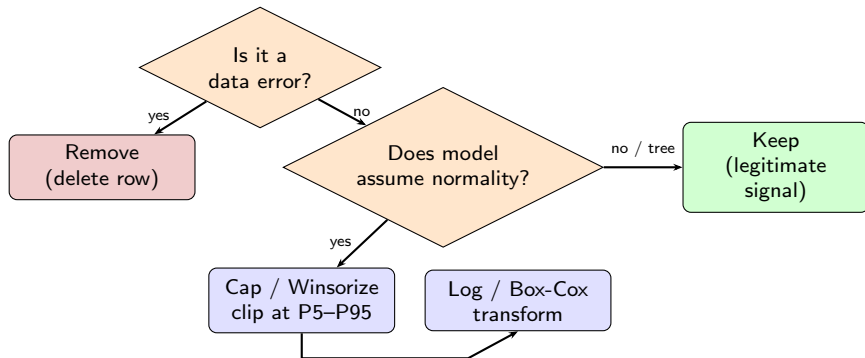
- **Z-score:**
 $|z_i| > 3$; $z_i = (x_i - \mu)/\sigma$
 — **assumes normality**
- **IQR Rule:**
 $x < Q_1 - 1.5 \cdot \text{IQR}$ or
 $x > Q_3 + 1.5 \cdot \text{IQR}$
 — **robust**
- **Modified Z-score:** uses **median** instead of mean; better for skewed data

Algorithmic Methods:

- **Isolation Forest:** random splits; **anomalies** isolated in fewer partitions
- **DBSCAN:** density-based; points in low-density regions are **outliers**
- **Domain rules:** values outside **business logic** (e.g., age < 0, price < 0)



Outlier Handling II: *Treatment Strategies*



Practical Guidance

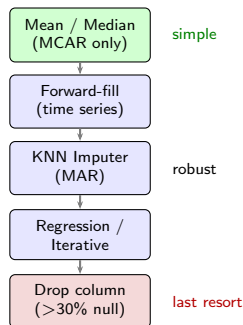
- **Always** investigate outliers before removing: they may encode the most valuable information.
- Tree-based models (RandomForest, XGBoost) are **naturally robust** to outliers — no treatment needed.

Missing Data: Mechanisms and Imputation

Missing Data Mechanisms

- **MCAR:** *Completely at Random* — absence **unrelated** to any variable; safest.
- **MAR:** *At Random* — depends on **observed** variables; imputation possible.
- **MNAR:** *Not at Random* — depends on the **missing value itself**; risk of bias.

Imputation Methods



Rule of Thumb

- < 5% missing → mean/median.
- 5–30% → model-based imputer.
- > 30% → **consider dropping** the column.

Feature Engineering

Definition

The process of transforming **raw data** into **informative features** that better expose the **underlying structure** to learning algorithms. Feature engineering often has **more impact** than algorithm choice.



Feature Engineering

Encoding categorical variables:

- **One-Hot:** binary indicator per category; sparse
- **Target encoding:** replace with conditional mean of target
- **Embeddings:** dense learned representation

Numerical transformations:

- **Scaling:** standardize ($\mu = 0, \sigma = 1$) or min-max $[0, 1]$
- **Log / Box-Cox:** reduce right-skewness
- **Polynomial:** x^2, x_1, x_2 to capture non-linearities

Domain-specific features:

- **Temporal:** day-of-week, holiday flag, season, lag features
- **Text (NLP):** TF-IDF, sentiment score, word counts
- **Aggregations:** "orders in last 30 days"; rolling mean / max
- **Ratios:** revenue-per-user, click-through-rate



Feature Engineering

Encoding categorical variables:

- **One-Hot:** binary indicator per category; sparse
- **Target encoding:** replace with conditional mean of target
- **Embeddings:** dense learned representation

Numerical transformations:

- **Scaling:** standardize ($\mu = 0, \sigma = 1$) or min-max $[0, 1]$
- **Log / Box-Cox:** reduce right-skewness
- **Polynomial:** x^2, x_1x_2 to capture non-linearities

Domain-specific features:

- **Temporal:** day-of-week, holiday flag, season, lag features
- **Text (NLP):** TF-IDF, sentiment score, word counts
- **Aggregations:** "orders in last 30 days"; rolling mean / max
- **Ratios:** revenue-per-user, click-through-rate



Feature Engineering

Encoding categorical variables:

- **One-Hot**: binary indicator per **category**; sparse
- **Target encoding**: replace with **conditional mean** of target
- **Embeddings**: dense **learned** representation

Numerical transformations:

- **Scaling**: standardize ($\mu = 0, \sigma = 1$) or min-max $[0, 1]$
- **Log / Box-Cox**: reduce **right-skewness**
- **Polynomial**: x^2, x_1x_2 to capture **non-linearities**

Domain-specific features:

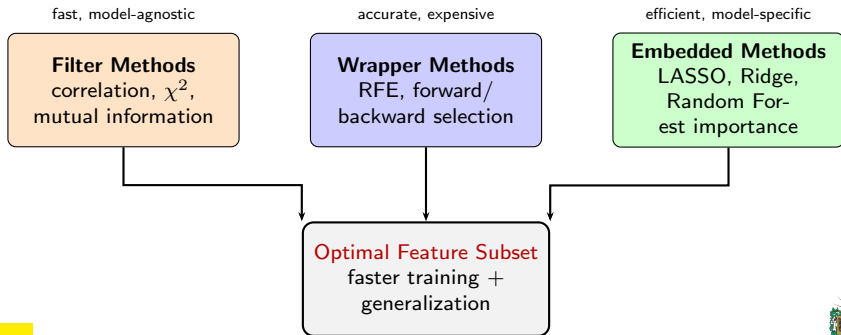
- **Temporal**: day-of-week, **holiday flag**, season, lag features
- **Text (NLP)**: TF-IDF, **sentiment score**, word counts
- **Aggregations**: “orders in last 30 days”; **rolling** mean / max
- **Ratios**: revenue-per-user, **click-through-rate**



Feature Selection

Why Select Features?

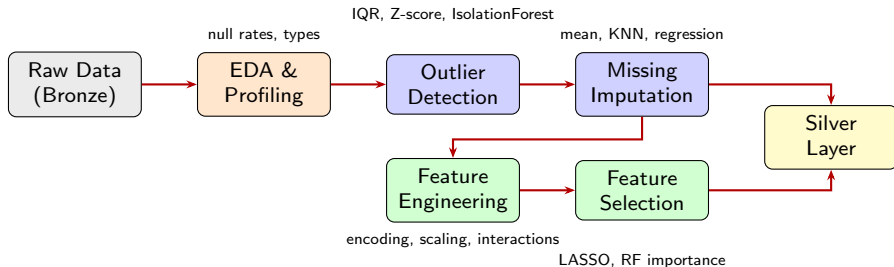
Reducing the **feature space** lowers **overfitting risk**, speeds up training, and improves **model interpretability**. **Irrelevant** or **redundant** features add noise.



Data Curation Pipeline: End to End

Data Curation

The systematic process of **discovering**, **cleaning**, **enriching**, and **organizing** data to make it **fit for purpose** for analysis and machine learning.



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift**: the statistical distribution of features changes over time — models become *stale*
- **Class imbalance**: rare events (fraud, churn) underrepresented — SMOTE, class weights
- **Leakage**: future information accidentally included in training features — invalidates evaluation
- **High cardinality**: millions of unique categories (e.g., IP) — hashing, embeddings required
- **Multicollinearity**: highly correlated features inflate variance in linear models — VIF analysis, PCA
- **Privacy**: personal data requires anonymization or differential privacy before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA

Privacy: personal data requires **anonymization** or **differential privacy** before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA

Privacy: personal data requires **anonymization or differential privacy** before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA

Privacy: personal data requires **anonymization or differential privacy** before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA

Privacy: personal data requires **anonymization** or **differential privacy** before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA

Privacy: personal data requires **anonymization** or **differential privacy** before curation



Data Curation: Current Challenges

Why Curation is Hard in Practice

Real-world data violates every textbook assumption simultaneously. **80% of data science work** is data preparation.

- **Concept drift:** the **statistical distribution** of features changes over time — models become *stale*
- **Class imbalance:** rare events (**fraud, churn**) underrepresented — SMOTE, class weights
- **Leakage:** future information **accidentally** included in training features — invalidates evaluation
- **High cardinality:** millions of unique categories (**user IDs**) — hashing, embeddings required
- **Multicollinearity:** highly **correlated features** inflate variance in linear models — VIF analysis, PCA
- **Privacy:** personal data requires **anonymization** or **differential privacy** before curation



Outline

- 1 ETL and Exploratory Data Analysis
- 2 Data Curation
- 3 Data Visualization and Visual Storytelling



Why Data Visualization?

Anscombe's Quartet (1973)

Four datasets with **identical** summary statistics ($\bar{x}$, $\bar{y}$, s_x , s_y , r) but **completely different** visual patterns. Statistics alone can **deceive**; visualization **reveals truth**.

Visualization enables:

- **Pattern recognition** faster than any table
- **Outlier detection** at a glance
- Communication of insights to **non-technical** audiences
- Monitoring of **data distributions** over time

Python ecosystem:

- **matplotlib**: low-level publication quality
- **seaborn**: statistical defaults on matplotlib
- **plotly**: interactive web-ready charts
- **bokeh**: real-time streaming dashboards



Why Data Visualization?

Anscombe's Quartet (1973)

Four datasets with **identical** summary statistics ($\bar{x}$, $\bar{y}$, s_x , s_y , r) but **completely different** visual patterns. Statistics alone can **deceive**; visualization **reveals truth**.

Visualization enables:

- **Pattern recognition** faster than any table
- **Outlier detection** at a glance
- Communication of insights to **non-technical** audiences
- Monitoring of **data distributions** over time

Python ecosystem:

- matplotlib: low-level **publication** quality
- seaborn: statistical **defaults** on matplotlib
- plotly: **interactive** web-ready charts
- bokeh: real-time **streaming** dashboards

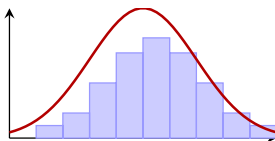


Types of Plots I: *Distribution and Comparison*

Distribution

- **Histogram:** shape, modality, **skewness** of a continuous variable; bin width critical
- **KDE:** smooth **density** estimate; area = 1
- **Box Plot:** median, IQR, **outliers** at a glance

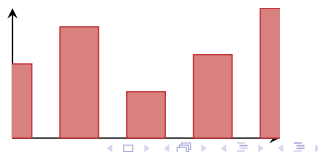
KDE + Histogram



Comparison

- **Bar Chart:** aggregate values per **category**; sorted for readability
- **Grouped Bar:** **multiple series** per category
- **Radar:** multi-axis comparison — use **sparingly**

Bar Chart



Types of Plots II: *Relationship and Composition*

Relationship

- **Scatter**: two continuous vars; add **regression** line to show trend
- **Bubble**: scatter + **size** encoding a 3rd variable
- **Heatmap**: pairwise **correlation** matrix or 2-D aggregates
- **Pair Plot**: all pairwise scatter plots — EDA for multivariate data

Composition & Time

- **Pie/Donut**: part-to-whole, use only with few categories (≤ 5)
- **Stacked Bar**: composition across categories
- **Line Plot**: trend over time, best for time-series data
- **Area Chart**: stacked lines encoding cumulative volumes



Types of Plots II: *Relationship and Composition*

Relationship

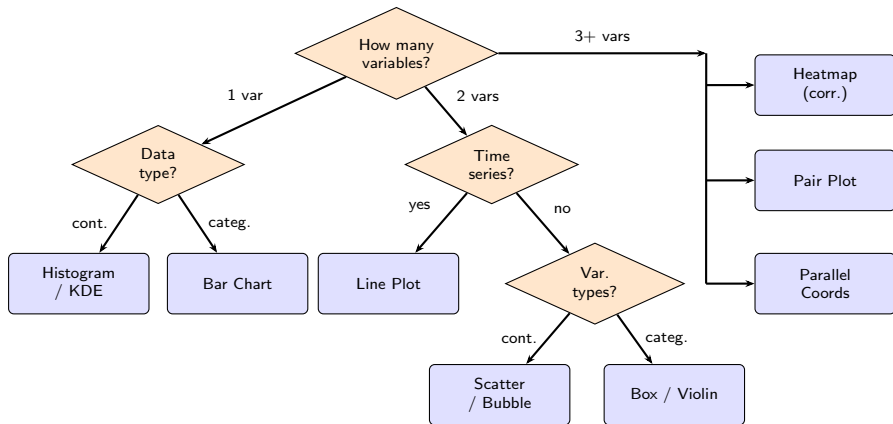
- **Scatter**: two continuous vars; add **regression** line to show trend
- **Bubble**: scatter + **size** encoding a 3rd variable
- **Heatmap**: pairwise **correlation** matrix or 2-D aggregates
- **Pair Plot**: all pairwise scatter plots — EDA for multivariate data

Composition & Time

- **Pie/Donut**: part-to-whole; use only with **few categories** (≤ 5)
- **Stacked Bar**: composition **across categories**
- **Line Plot**: **trend over time**; best for time-series data
- **Area Chart**: stacked lines encoding **cumulative volumes**



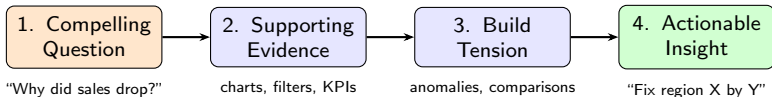
Choosing the Right Plot: *Decision Guide*



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** purposeful, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units always
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

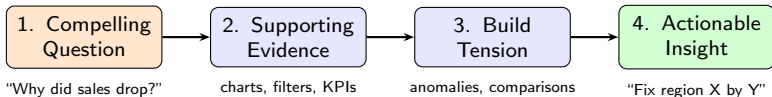
- **Executive:** KPI cards, single headline number
- **Analyst:** drill-down, filters, raw data access
- **Public:** plain language, zero jargon



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** **purposeful**, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units **always**
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

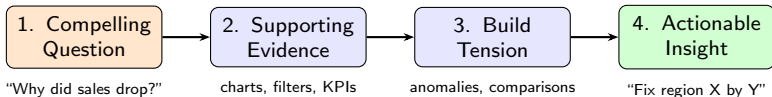
- Executive: KPI cards, single headline number
- Analyst: drill-down, filters, raw data access
- Public: plain language, zero jargon



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** **purposeful**, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units **always**
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

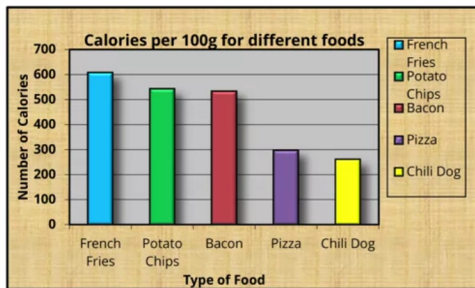
- **Executive:** KPI cards, single **headline number**
- **Analyst:** drill-down, **filters**, raw data access
- **Public:** **plain language**, zero jargon



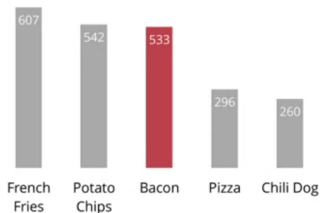
Simplicity is Key: Data-Ink Ratio

Data-Ink Ratio (Tufte, 1983)

The proportion of **non-erasable** ink used to represent data compared to the total ink used in the graphic. Higher ratio → more efficient communication.



Calories per 100g



Some color visualization rules

Color Visualization Rules

Use color **purposefully**, not decoratively; ensure colorblind-safe palettes.



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear objectives; avoid *information overload*
- Prioritize top-left placement (F-pattern reading)
- Use **color**; large, high-contrast numbers
- Support *drill-down* and *drill-up*
- Refresh *data* at appropriate cadence
- Separate *technical* from *functional* dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

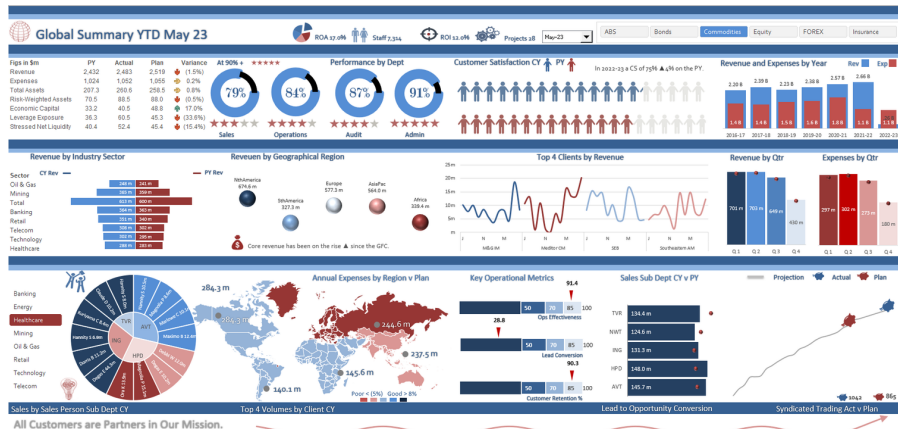
A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards

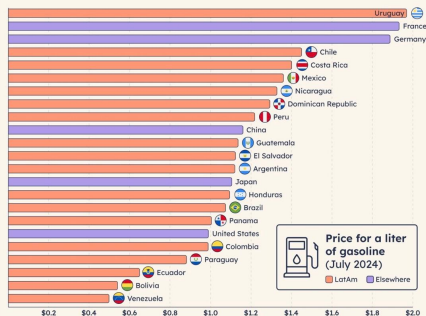


Big Dashboard Example



Insights from a “Big Picture”

The price of gas in Latin America



Source: GlobalPetrolPrices.com

Latinometrics



Insights from a “Big Picture”

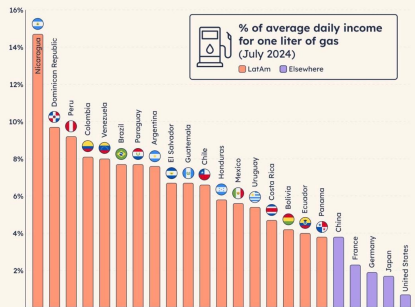
The price of gas in Latin America



Source: GlobalPetrolPrices.com

Latinometrics

How unaffordable is gas in Latin America?



Source: Numbeo, GlobalPetrolPrices.com

Latinometrics



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`, `html.Div()`, `dcc.Dropdown()` → reactive UI tree
- **Callbacks:** `@app.callback()` → input → output reactive updates
- **Bootstrap:** `dash-bootstrap-components` or `reactive-gridlayout`

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts → audience: data engineer
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns → audience: functional user



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`,
`html.Div()`,
`dcc.Dropdown()` — reactive
UI tree
- **Callbacks:** `@app.callback()`
— **input** → **output** reactive
updates
- **Bootstrap:**
`dash-bootstrap-components`
for **responsive** grid layout

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts — audience: data engineer
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns — audience: functional user



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`, `html.Div()`, `dcc.Dropdown()` — reactive **UI tree**
- **Callbacks:** `@app.callback()` — **input** → **output** reactive updates
- **Bootstrap:** `dash-bootstrap-components` for **responsive** grid layout

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts — audience: **data engineer**
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns — audience: **functional user**



Current Challenges in Data Visualization

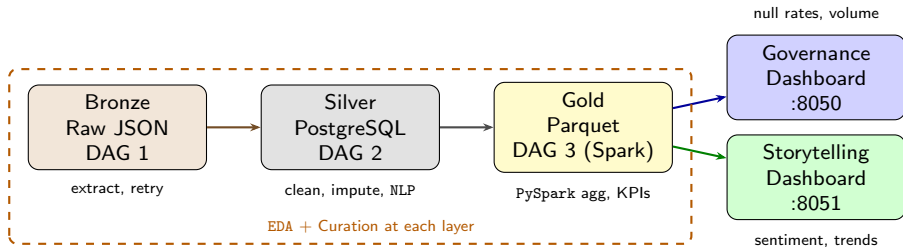
Visualization at Scale

Rendering **billions of data points**, maintaining **responsiveness** with streaming data, and ensuring **accessibility** are engineering challenges that go beyond picking chart types.

- **Scale:** rendering 10^9 points without lag — Deck.gl, WebGL, **data aggregation** before render
- **Dimensionality:** 10+ features in 2D — UMAP, t-SNE, **parallel coordinates**
- **Real-time:** maintaining **responsiveness** with streaming data (Kafka → WebSocket → dashboard)
- **Accessibility:** **colorblind-safe** palettes (viridis, cividis), screen reader support
- **Misleading visuals:** truncated axes, cherry-picked windows, inappropriate chart types — **data literacy** matters



Workshop Connection: Full Pipeline to Dashboard



Key Takeaway

Every slide in this module maps to a **concrete deliverable**: ETL → DAGs, EDA → Silver notebooks, Curation → Silver DAG transforms, Visualization → **Plotly Dash dashboards** (Workshops 3 & 4).



Outline

- 1 ETL and Exploratory Data Analysis
- 2 Data Curation
- 3 Data Visualization and Visual Storytelling



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

